TUTORIAL · SETUP & BEST PRACTICES

The Claude Code Handbook

VOLUME 1

Step-by-step guide to configuring Claude and working with it like a pro. Copy-paste ready, interface-agnostic.

AUDIENCE	Solo devs, teams, beginner to intermediate
FORMAT	8 chapters · 30 min to start · copy-paste appendix
COVERS	Terminal CLI · VS Code · Cursor · JetBrains · Desktop · Web
VERSION	V1 · May 20, 2026

Foreword

This guide exists because a well-configured Claude has nothing to do with a default Claude. Three hours of setup saves you six months of frustration.

Who this is for

- **The solo dev** coding with Claude every day, tired of re-explaining the same things.
- **The team** that wants Claude to behave consistently across everyone's machine.
- **The beginner moving past "vibe coding"** toward a reproducible workflow.

What this is NOT

- A *prompt engineering* course (writing prompts well).
- Claude API documentation (no Python with `anthropic.messages.create()`).
- A programming tutorial. You're expected to know how to run `git commit` and edit a file.

What you'll learn

- How to configure the **permanent context** Claude reads every session (`CLAUDE.md`).
- How to automate **repetitive actions** via hooks (zero tokens, deterministic).
- How to create **specialized roles** (sub-agents) that do one thing well.
- How to write custom **skills & slash commands** that wrap your workflows into one line.
- How to set up a **Coach mode** that reminds you of best practices at the right moment.
- How to maintain all this over time, without config debt.

Reading conventions

- 📋 **Copy-paste block** : everything in a box with a green left border is ready to paste as-is (adapting paths/names to your project).
- 💡 **Tip** : a shortcut or trick learned by practicing.
- ⚠️ **Warning** : a pitfall that costs time.
- 🚫 **Anti-pattern** : something people do that always ends badly.

> 💡 **HOW TO READ THIS GUIDE**
>
> Read chapters 0–2 in one go (≈ 20 min) and you'll already have a transformed Claude. Chapters 3–7 can be spread over 4 weeks at one per week, as the original cadence suggests. Keep the appendix open during setup for copy-paste.

Contents

00 Choose your interface

Before configuring anything, know where you're installing it. All Claude Code surfaces support the same mechanisms — but Claude Code is distinct from the Claude.ai chat product.

The 6 official Claude Code surfaces

All read the same `.claude/` config in your repo. You can switch between them without re-configuring anything.

SURFACE	WHAT'S SPECIAL
Terminal CLI	The most complete. Run `claude` in any project folder. Ideal for scripting, pipes, CI.
VS Code extension	Inline diffs, @-mentions, plan review. **Also works in Cursor** (official Anthropic extension, not `.cursorrules`).
JetBrains plugin	IntelliJ, PyCharm, WebStorm, etc. Available on the official JetBrains Marketplace.
Desktop app	Standalone macOS/Windows app. Visual diff, parallel sessions, scheduled tasks, cloud sessions.
Web	`claude.ai/code` in your browser. Ideal for long-running tasks, repos you don't have locally, mobile.
iOS app	Claude app for iPhone. Continue a session from your phone.

> 💡 **SHARED ENGINE**
>
> Every surface connects to the same Claude Code engine. Your `CLAUDE.md`, hooks, sub-agents, and `.mcp.json` work everywhere. Start a task in the terminal and resume it on your phone via the session.

Recommendations by profile

Solo dev working in an IDE

→ **VS Code or JetBrains extension** for everyday work, **CLI** in the terminal for scripted workflows. Full setup possible.

Team

→ Any surface, with a `.claude/` setup versioned in git. Everyone inherits the config.

Mobile / away from desk

→ **Web** on phone or **iOS app**. Kick off a long task, resume it on your machine with `claude --teleport`.

Cursor user

→ Install the official Claude Code extension in Cursor (`cursor:extension/anthropic.claude-code`). All `.claude/` config works.

> ⚠️ **CLAUDE.AI (CHAT) ≠ CLAUDE CODE**
>
> Claude.ai (the chat product) supports "Projects" with custom instructions (a lightweight CLAUDE.md equivalent), but **not** hooks, sub-agents, or MCP servers. This guide covers Claude Code (the 6 surfaces above). For chat alone, only chapters 1–2 (principles + CLAUDE.md) apply.

01 Philosophy — configure rather than vibe-code

The difference between a dev who uses Claude and a dev who gets 10× more out of it isn't the prompt. It's what happens *around* the prompt.

The vibe-coding problem

When you use Claude without configuration, here's what happens every session:

1. You re-explain your stack ("we're on Next.js 16, beware the revalidate breaking change...").
2. You re-explain your conventions ("camelCase for functions, kebab-case for files...").
3. Claude heads in a direction, you correct it ("no, we don't use alert(), it's a Sonner toast").
4. Claude runs a dangerous command, you approve 4 popups for trivial things.
5. You forget to run `pnpm verify` before a push, Vercel crashes.
6. You do the same sequence tomorrow.

Multiply by 250 sessions/year. You realize you've spent weeks doing **disposable context**.

The core idea

> *Anything you tell Claude more than 3 times should live in a file. Anything Claude does more than 3 times without human intervention should be a hook.*

The 3 principles of the Claudeur

> **PRINCIPLE 1 — CAPTURE CONTEXT**
>
> Stack, conventions, gotchas, project structure → `CLAUDE.md` . Claude reads it every session, automatically. You no longer have to explain "we're on Next.js 16".

PRINCIPLE 2 — DETERMINIZE THE AUTOMATABLE

Auto-format after each edit, log commands, session-start recaps → **hooks**. These are scripts (bash, python) that run on events. Zero tokens, zero friction.

PRINCIPLE 3 — SPECIALIZE ROLES

Security audit, formula validation, component scaffolding → **sub-agents**. Each sub-agent has its own context, model, and tools. Run a security audit without polluting the main session.

Time invested vs time saved

SETUP	TIME TO SET UP	GAIN PER SESSION	BREAK-EVEN
Basic CLAUDE.md	15 min	2–5 min	3 sessions
Permissions allowlist	10 min	1–3 popups avoided	1 session
Defensive PreToolUse hook	20 min	0 (but saves 1 catastrophe/year)	1st catastrophe avoided
PostToolUse format hook	15 min	30 seconds / Write	3 sessions
5 focused sub-agents	2 h	10–30 min / audit	~10 audits
3 slash commands	45 min	2 min / invocation	~20 calls

Total setup: **half a day**. Estimated gain: **15–30 min per active dev day**. Over 200 days/year, that's 50–100 hours saved.

02 The minimum viable setup — day 1

Goal: transform your Claude in 30 minutes. No prerequisites. Just 3 files to create.

Step by step

1 Install Claude Code

Pick your OS and surface:

```bash
# macOS, Linux, WSL — native install (recommended, auto-updates)
curl -fsSL https://claude.ai/install.sh | bash

# Windows PowerShell
irm https://claude.ai/install.ps1 | iex

# macOS via Homebrew
brew install --cask claude-code

# Windows via WinGet
winget install Anthropic.ClaudeCode
```

Then, in any project folder:

```bash
cd your-project
claude
```

For IDEs: Extensions → search "Claude Code" (VS Code, Cursor, JetBrains). For the browser: `claude.ai/code`. For the app: download from claude.ai. All these surfaces read the same `.claude/` files.

2 Create `CLAUDE.md` in your project root

It's the brain of your project. Claude loads it automatically every session. Minimal template:

CLAUDE.MD

```
# <Project name>

One-sentence description of what the app does.

## Tech stack
- Language: <Python 3.12 | TypeScript 5 | Go 1.22 | ...>
- Framework: <Next.js 16 | FastAPI | Rails | ...>
- DB: <Postgres via Supabase | SQLite | MongoDB | ...>
- Tests: <pytest | vitest | jest>
- Deployment: <Vercel | Fly.io | Railway | ...>

## Useful commands
- `` — start the local server
- `` — type-check before push (what blocks deploy)
- `` — run the test suite

## Conventions
- <Naming rule> (e.g., camelCase JS, kebab-case files)
- <Structure rule> (e.g., every page has its own `actions.ts`)
- <Behavior rule> (e.g., never `alert()`, always Sonner toast)

## Gotchas
- <Known stack pitfall> (e.g., Next.js 16 changed `revalidate`)
- <Internal project pitfall>

## Git workflow
- Target branch: `<master | main>`
- Commit format: `type(scope): description`
- <PR or direct push>

## Hard rules
- **Never** <X> without confirmation
- **Always** <Y> before <Z>
```

3 Create `.claude/settings.json`

The file that controls permissions and hooks. Minimal template that eliminates 80 % of popups:

.CLAUDE/SETTINGS.JSON

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read",
      "Grep",
      "Glob",
      "Bash(git status:*)",
      "Bash(git diff:*)",
      "Bash(git log:*)",
      "Bash(git show:*)",
      "Bash(git branch:*)",
      "Bash(ls:*)",
      "Bash(cat:*)",
      "Bash(find:*)",
      "Bash(head:*)",
      "Bash(tail:*)",
      "Bash(wc:*)"
    ],
    "deny": [
      "Bash(rm -rf*)",
      "Bash(sudo*)",
      "Bash(curl* | bash*)"
    ]
  }
}
```

Adapt the `allow` list to your stack (add `"Bash(pnpm verify)"`, `"Bash(npm test:*)"`, etc.).

4 Add `.claude/` to `.gitignore` (selectively)

You want to version team config but not local logs. Add to your `.gitignore`:

.GITIGNORE

```
# Claude Code — version agents/commands/hooks/settings.json
# but not logs or personal overrides
.claude/*
!.claude/agents/
!.claude/commands/
!.claude/hooks/
!.claude/settings.json
.claude/logs/
.claude/settings.local.json
```

5 Test

Launch Claude Code. Ask a question: *"Read CLAUDE.md and summarize the stack in 3 lines."* If it answers correctly with what you put in the file → ✅ it works. Otherwise: verify the file is in the project root, not in `src/` or elsewhere.

> 💡 **AUTO-INIT**
>
> Claude Code includes an `/init` command that scans your repo and proposes a first version of CLAUDE.md. Run it, then adjust: remove what's wrong, add your specific rules.

> 🚫 **ANTI-PATTERN**
>
> Putting 800 lines in CLAUDE.md on day 1. The file must stay **stable and readable**. If you find yourself scrolling for info, it's time to split (cf. chapter 7).

What you gained in 30 minutes

- No more re-explaining the stack or conventions.
- No more popups for standard git/ls/cat commands.
- Dangerous commands (`rm -rf`, `sudo`) are blocked.
- The repo is ready for the next levels.

03 Hooks — week 1

A hook is a script triggered by an event. Zero tokens, zero Claude intervention, zero friction. It's the "deterministic automation" layer Claude doesn't even see.

The 4 key events (out of ~25)

Claude Code exposes about 25 events (UserPromptSubmit, SessionEnd, SubagentStart/Stop, FileChanged, PreCompact, Notification, etc.). This guide covers the **4 that handle 80 % of needs**. The full list is in the official docs at `code.claude.com/docs/en/hooks`.

EVENT	WHEN	TYPICAL USE CASES
`SessionStart`	When Claude starts	Git recap, silent fetch, project state
`PreToolUse`	Before each tool call	Block `rm -rf`, writes to `.env`, `--force` pushes
`PostToolUse`	After each tool call	Auto-format, activity log
`Stop`	When Claude finishes its turn	System notif, Coach mode, Slack/Discord ping

Hook #1 — PostToolUse: auto-format after each write

Create `.claude/hooks/post-edit-format.sh`:

BASH

```
#!/usr/bin/env bash
# Auto-format TS/JS files after Write/Edit via ESLint --fix.
# Adapt the formatter to your stack: prettier, black, gofmt, rustfmt...
set -e

INPUT="$(cat 2>/dev/null || echo '{}')"
FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
[ -z "$FILE" ] || [ ! -f "$FILE" ] && exit 0

case "$FILE" in
  *.ts|*.tsx|*.js|*.jsx|*.mjs) ;;
  *) exit 0 ;;
esac

# Ignore node_modules, build, migrations
case "$FILE" in
  *node_modules*|*.next/*|*build/*|*dist/*) exit 0 ;;
esac
```

```
cd "$(dirname "$0")/../.."
pnpm exec eslint --fix "$FILE" >/dev/null 2>&1 || true
exit 0
```

Don't forget `chmod +x .claude/hooks/post-edit-format.sh`.

Hook #2 — PreToolUse: defensive guard

This hook is **the most important**. It blocks destructive commands before they run. Create `.claude/hooks/pre-tool-guard.sh`:

BASH

```
#!/usr/bin/env bash
# Guards: block destructive actions before execution.

INPUT="$(cat 2>/dev/null || echo '{}')"
TOOL=$(echo "$INPUT" | jq -r '.tool_name // empty' 2>/dev/null)

# 1. Block any write to a .env*
if [ "$TOOL" = "Edit" ] || [ "$TOOL" = "Write" ]; then
  FILE=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty' 2>/dev/null)
  case "$FILE" in
    *.env|*.env.*|.env)
      echo "BLOCKED: .env file protected. Use provider env vars." >&2
      exit 2
      ;;
  esac
fi

# 2. Block dangerous Bash
if [ "$TOOL" = "Bash" ]; then
  CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty' 2>/dev/null)

  # rm -rf on dangerous paths
  if echo "$CMD" | grep -qE 'rm[[:space:]]+-[a-zA-Z]*[rf][a-zA-Z]*[[:space:]]+
(/|~|\.\./|\$HOME)'; then
    echo "BLOCKED: rm -rf on dangerous path." >&2
    exit 2
  fi

  # git push --force
  if echo "$CMD" | grep -qE 'git[[:space:]]+push.*(--force|--force-with-lease|[[:space:]]-
f([[:space:]]|$))'; then
    echo "BLOCKED: git push --force. Ask user for explicit confirmation." >&2
    exit 2
  fi

  # git commit --no-verify (skip pre-commit hooks)
  if echo "$CMD" | grep -qE 'git[[:space:]]+commit.*--no-verify'; then
    echo "BLOCKED: --no-verify skips checks. Fix errors first." >&2
    exit 2
  fi
fi

exit 0
```

Hook #3 — SessionStart: project recap

When a session opens, show the project state in 5 lines. Create `.claude/hooks/session-start.sh`:

BASH

```
#!/usr/bin/env bash
# Session recap: branch, dirty files, ahead/behind commits, last 3 commits.
set -e
cd "$(dirname "$0")/../.."

git fetch origin --quiet 2>/dev/null || true

BRANCH=$(git rev-parse --abbrev-ref HEAD 2>/dev/null || echo "?")
DIRTY=$(git status --porcelain 2>/dev/null | wc -l | xargs)
AHEAD=$(git rev-list --count @{u}..HEAD 2>/dev/null || echo "?")
BEHIND=$(git rev-list --count HEAD..@{u} 2>/dev/null || echo "?")

echo ""
echo "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"
echo "  <Project> · session started · $(date '+%a %d %b %Hh%M')"
echo "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"
echo "  Branch       : $BRANCH"
[ "$DIRTY" = "0" ] && echo "  Working tree : clean" || echo "  Working tree : $DIRTY file(s)
modified"
[ "$AHEAD" != "0" ] && [ "$AHEAD" != "?" ] && echo "  ⚠  $AHEAD unpushed commit(s)"
[ "$BEHIND" != "0" ] && [ "$BEHIND" != "?" ] && echo "  ⚠  $BEHIND commit(s) behind origin"
echo ""
echo "  Last commits:"
git log --oneline -3 2>/dev/null | sed 's/^/    /'
echo "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"
echo ""
```

Activate hooks in `settings.json`

.CLAUDE/SETTINGS.JSON

```
{
  "permissions": { /* ... see chapter 2 ... */ },
  "hooks": {
    "SessionStart": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/session-start.sh" }
      ]}
    ],
    "PreToolUse": [
      { "matcher": "Edit|Write|Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
      ]}
    ],
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": ".claude/hooks/post-edit-format.sh" }
      ]}
    ]
  }
}
```

⚠️ **PREREQUISITE**

The bash hooks above use `jq` to parse the JSON payload. Install it if you don't have it: `brew install jq` (macOS) or `apt install jq` (Linux). Otherwise, write hooks in Python — equivalent in 5 lines with `json.load(sys.stdin)`.

Test a hook without restarting the session

BASH

```
# Simulate a PreToolUse event
echo '{"tool_name":"Bash","tool_input":{"command":"rm -rf /"}}' \
  | .claude/hooks/pre-tool-guard.sh
echo "Exit code: $?"
# Should print "BLOCKED: rm -rf..." and exit 2
```

💡 **PERFORMANCE**

A hook should run in **under 200 ms**. Beyond that you'll feel friction. If your hook makes network calls or runs a heavy linter, mark it `"async": true` in the config — it runs in the background without blocking.

04 Sub-agents — week 2

A sub-agent is a specialized role with its own context, model, tools, and system prompt. You trigger it for a precise task and it returns a structured result.

Why split rather than do everything in one

1. **Context preserved** — each agent starts with a clean context, doesn't pollute the main session.
2. **Narrow scope** — an audit agent has no Write/Edit tools. It *cannot* break code.
3. **Right-sized model** — a complex audit uses `opus`, a quick classification uses `haiku`. You optimize cost.
4. **Specialization** — the security agent knows OWASP; the domain agent knows your conventions. Not mixed.

Anatomy of an agent

An agent is a markdown file at `.claude/agents/<name>.md` with a YAML frontmatter and a system prompt in markdown.

.CLAUDE/AGENTS/CODE-AUDITOR.MD

```
---
name: code-auditor
description: >
  READ-ONLY code quality audit. Use proactively when the user says
  "audit the code", "review", or before a large refactor.
  Detects: dead code, duplication, complexity, anti-patterns, unjustified
  `any` types. Modifies NO files — produces a structured report.
model: opus
tools: Read, Grep, Glob, Bash, WebFetch
---

You are a senior code auditor for project <name>. You work read-only.

## Project context
- Stack: <quick summary>
- Conventions to respect (cf. CLAUDE.md): <quick summary>

## Your mission
For each audited file, identify:
1. **Dead code**: unused functions, variables, imports
2. **Duplication**: repeated patterns that should be factored
3. **Complexity**: functions > 30 lines, conditionals nested > 3 deep
4. **Anti-patterns**: misused useEffects, non-parallel queries, `any` types

## Output format
For EACH finding:
```

```
- **Severity**: 🔴 / 🟠 / 🟡 / 🟢
- **File:line**
- **Observation** + **Impact** + **Recommended fix**

End with: Overall score /10 + Top 3 priority actions.

## Rules
- Read-only: no Edit, no Write.
- Don't nitpick style if already covered by the linter.
- Prioritize what has real impact on maintainability.
```

Pick a model

The `model` field accepts an alias (`sonnet`, `opus`, `haiku`), a full ID (e.g., `claude-opus-4-7`), or `inherit` (default). Exact pricing varies; in practice sonnet and opus cost several times more than haiku.

MODEL	WHEN
`haiku`	Classification, quick triage, codebase exploration, repetitive tasks
`sonnet`	Code generation, standard audits, structured content — the sane default
`opus`	Complex audits (security, DB, architecture), long reasoning, strategic decisions

Rule of thumb: any audit that could lose data or compromise production → opus. Everything else → sonnet by default. Haiku when speed beats depth.

The `effort` field controls reasoning effort (separate from model). Values: `low`, `medium`, `high`, `xhigh`, `max` — available levels depend on the model.

Restrict tools (least privilege)

Give only the tools the agent needs. A read-only auditor never gets Write/Edit/dangerous Bash.

Audit agent (read-only)

`tools: Read, Grep, Glob, Bash, WebFetch, WebSearch`
No Write/Edit.

Agent that creates code

`tools: Read, Write, Edit, Bash, Glob`
Bash only if needed (tests).

Agent that writes content

`tools: Read, Write, Edit, Grep, WebFetch, WebSearch`
No Bash.

Classification agent

`tools: Read, Grep, Glob`
Minimalist, haiku model.

The "Use proactively when..." description

Claude reads each agent's `description` and decides automatically when to invoke. The more precise the description, the more accurate the invocation.

FRONTMATTER

```
description: >
  <What the agent does in one sentence>.
  Use proactively when user says "<keyword 1>", "<keyword 2>", "<keyword 3>",
  or before <trigger event>.
  Covers: <list of what's covered>.
  <"Read-only" or "Modifies files in X">.
```

How many agents max

⚠️ **THE AGENT FACTORY**

Past **10–12 agents**, you don't remember what each one does. You end up using the same 3. Keep the folder lean: 5–8 sharp agents > 20 forgotten ones.

You can start with these 3 agents that cover 80 % of cases:

1. `code-auditor` — quality audit (template above)
2. `security-auditor` — RLS, secrets, OWASP, GDPR (cf. appendix F)
3. `qa-tester` — end-to-end user flows after a large change

Add more as you identify a *recurring* need.

Invoking a sub-agent

- **Automatic** — Claude reads the `description` and triggers the agent when the pattern matches.
- **Explicit** — the user says *"Launch security-auditor on this week's server actions"*.
- **Via a slash command** — which wraps the invocation with pre-formatted context (cf. chapter 5).

> 💡 **TEST AN AGENT**
>
> After creation, invoke it once on a real case. Read the report. If it misses obvious things or its tone isn't right: tune the system prompt. Iterate 2–3 times before considering the agent stable.

05 Skills & slash commands — week 3

A skill (or slash command) is a shortcut that gives you a workflow by typing `/<name>`. It's your "reusable playbooks" layer.

> ⚠️ **SKILLS + COMMANDS MERGER**
>
> As of late 2025, "custom commands" and "skills" are **merged** in Claude Code. A `.claude/commands/deploy.md` file and a `.claude/skills/deploy/SKILL.md` skill both create `/deploy` and behave the same. Your existing `.claude/commands/` files still work (backwards compatible). The recommended modern path is `.claude/skills/` — it unlocks: supporting files alongside, invocation control, and Claude loading the skill automatically when relevant.

Skill vs sub-agent vs hook

	SKILL / SLASH COMMAND	SUB-AGENT	HOOK
Trigger	You type `/name` or Claude auto-loads	Auto or explicit	Event (auto)
Context	Current session	New isolated context	N/A (script)
Tokens	Yes	Yes (new context overhead)	**Zero**
Best for	Recurring workflow, doc/checklist	Isolated expertise, heavy task	Deterministic automation

Two ways to create one

Path A — Simple: `.claude/commands/<name>.md`

A single markdown file with frontmatter + instructions. Covers 90 % of cases.

.CLAUDE/COMMANDS/SHIP.MD

```
---
description: "Verify + commit + push in one command"
argument-hint: "<commit message>"
```

```
---

# /ship — Verify, commit & push

## Procedure

### 1. Pre-check
Run the typecheck command (e.g. `pnpm verify`). If red → STOP, show errors,
ask the user whether to fix or abandon. Never commit with red typecheck.

### 2. Git state
Run `git status --short` and `git diff --stat`. Show modified files and diff
summary. If zero files modified → "Nothing to commit" and stop.

### 3. Build the commit
Message = `$ARGUMENTS`. If empty or < 10 chars, ask for a clearer message.
Format to project style: `type(scope): summary` (cf. CLAUDE.md).

### 4. Stage + commit + push
In this order:
1. `git add` specific files (not blind `git add .`)
2. `git commit -m "..."` with the built message
3. `git push origin <target branch>`

### 5. Confirmation
Display SHA, message, insertions/deletions summary.

## Safeguards
- **Never** use `--no-verify` unless explicitly requested.
- **Never** force-push without explicit confirmation.
- If push fails (rejected): pull `--no-rebase`, resolve conflicts, re-push.
```

Path B — Powerful: `.claude/skills/<name>/SKILL.md`

A dedicated folder with a `SKILL.md` + supporting files (scripts, references, examples). Best when the workflow uses bundled scripts or detailed docs.

STRUCTURE

```
.claude/skills/deploy/
├── SKILL.md          ← Main instructions (required)
├── reference.md      ← Detailed doc (loaded on demand)
└── scripts/
    └── verify.sh     ← Executable script
```

.CLAUDE/SKILLS/DEPLOY/SKILL.MD

```
---
name: deploy
description: Deploy to production with verify + commit + push
disable-model-invocation: true
allowed-tools: Bash(pnpm verify) Bash(git *)
---

# /deploy — Production deploy

Use the script bundled with this skill:

```bash
bash ${CLAUDE_SKILL_DIR}/scripts/verify.sh
```
```

```
Then commit and push following the standard workflow...
```

Key skill frontmatter fields (all optional except `description`):

- `description` — when Claude should use the skill (recommended)
- `disable-model-invocation: true` — only the user can invoke (recommended for `/deploy`, `/ship`, anything with side effects)
- `allowed-tools` — pre-authorized tools while the skill runs
- `argument-hint` — autocomplete hint (e.g. `<commit message>`)
- `context: fork` — run in an isolated sub-agent
- `paths` — glob patterns that limit when the skill auto-activates

3 commands to code in week 3

`/ship "<message>"` — deploy in one line

Template above. It's **the most useful command**. Replaces your manual commit/push routine and guarantees the typecheck passes.

`/audit-quick` — code + security audit in parallel

.CLAUDE/COMMANDS/AUDIT-QUICK.MD

```
---
description: "Quick audit: code + security in parallel, synthesized report"
---

# /audit-quick — Pre-release audit

Launch these 2 agents IN PARALLEL (same message, multiple tool calls):

**Agent code-auditor**:
> Audit code, focus: dead code, duplication, anti-patterns,
> unjustified `any`. Report ≤ 500 words ranked 🔴/🟠/🟡.

**Agent security-auditor**:
> Audit security, focus: RLS, secrets, XSS, PII exposure, OWASP.
> Report ≤ 500 words ranked 🔴/🟠/🟡.

When both finish, present a unified report:
- 🔴 Blockers
- 🟠 Important
- 🟢 OK

End with a single priority recommendation.
```

`/standup` — daily recap

.CLAUDE/COMMANDS/STANDUP.MD

```
---
description: "Morning recap: 24h commits, WIP, suggested focus"
---

# /standup

Launch in parallel:
- `git log --since='24 hours ago' --oneline`
- `git status --short` + `git diff --stat`

Report in 3 sections, ≤ 200 words:
- 🟢 Done yesterday
- 🟡 In progress (WIP)
- 🎯 Today's focus (suggestion)

End with a question: "Tackling focus #1 or something else?"
```

The `$ARGUMENTS` pattern

When the user types `/ship "fix: header typo"`, the string in quotes is available in the skill via `$ARGUMENTS`. Use it to customize behavior. Indexed forms (`$0`, `$1`, `$ARGUMENTS[N]`) access individual arguments.

> 💡 **DISAMBIGUATION**
>
> Create a `.claude/COMMANDS.md` file (outside the `commands/` folder — putting it inside would create a `/COMMANDS` slash command) listing all your commands with a "I want X → command Y" table. Saves time when you have 10+ commands.

> 🚫 **ANTI-PATTERN**
>
> Putting a `README.md` inside `.claude/commands/`. Claude Code loads **every** `.md` in this folder as a slash command. You'd end up with a `/README` command. Put docs outside the folder.

06 Coach mode — week 4

Coach mode = a system that reminds you of best practices at the right moment, without you having to think about it. Three complementary layers: CLAUDE.md rules, observing Stop hook, manual `/coach` slash command.

Why a Coach

Once your setup is in place, you accumulate useful slash commands (`/ship`, `/audit-quick`, `/loro`, etc.). But in the heat of the moment, you forget to use them. You end up running `git commit + git push` manually, no verify, and Vercel crashes.

The Coach fixes this in 3 layers.

Layer 1 — Rules in CLAUDE.md

Add a *"Coach mode"* section in `CLAUDE.md` with a trigger table. I re-read it every session.

CLAUDE.MD

```
## Coach mode — when to propose what

Triggers to honor before acting. If a condition is true,
propose the command/agent rather than just running.

| Observed trigger | Action to propose |
|---|---|
| Task touches ≥ 5 files or new transversal feature | `/new-feature <slug>` |
| UI widget modification | `/loro <file>` before ship |
| Critical calculation modification | `/trading-check <fn>` after change |
| SQL migration | `db-schema-reviewer` agent before push |
| Server action / auth route modified | `security-auditor` agent before push |
| ≥ 10 files modified since last commit | `/audit-quick` before ship |
| Before any `git push` | `/ship "<message>"` |

**General rule**: if a command covers ~80 % of what was about to be
done manually, propose first. The user can always reply "no, do it
manually" — that's fine.
```

Layer 2 — Stop hook that observes

The `Stop` hook fires when Claude finishes its turn. It reads the current session's activity log, detects what was touched, and prints targeted suggestions **in your terminal**, consuming zero tokens.

Prerequisite: have a PostToolUse hook that logs Write/Edit/Bash to `.claude/logs/activity.log` (cf. appendix D). Then create `.claude/hooks/coach-suggest.sh` :

BASH

```
#!/usr/bin/env bash
# Stop hook — Coach. Analyzes the current session's activity-log
# and prints suggestions based on what was touched.
set -e
cd "$(dirname "$0")/../.."

# Coach mute: if flag exists, exit silently
[ -f ".claude/coach-mute" ] && exit 0

LOG=".claude/logs/activity.log"
[ ! -f "$LOG" ] && exit 0

INPUT="$(cat 2>/dev/null || echo '{}')"
SID=$(echo "$INPUT" | jq -r '.session_id // empty' | cut -c1-8)
[ -z "$SID" ] && exit 0

RECENT=$(grep "\[$SID\]" "$LOG" 2>/dev/null | tail -200)
[ -z "$RECENT" ] && exit 0

FILES=$(echo "$RECENT" | grep -E 'WRITE|EDIT' | sed 's/.*| //' | sort -u)
WRITES=$(echo "$RECENT" | grep -cE 'WRITE|EDIT')
[ "$WRITES" -eq 0 ] && exit 0

SUGGESTIONS=""

# Trigger 1: widget touched → /loro
if echo "$FILES" | grep -q "components/.*widget"; then
  W=$(echo "$FILES" | grep "components/.*widget" | head -1)
  SUGGESTIONS="${SUGGESTIONS}  • /loro $W\n      → polish the modified widget\n"
fi

# Trigger 2: auth / API touched → security-auditor
if echo "$FILES" | grep -qE "actions\.ts|/auth/|/api/"; then
  SUGGESTIONS="${SUGGESTIONS}  • Agent security-auditor\n      → check RLS / secrets on touched
routes\n"
fi

# Trigger 3: ≥ 2 writes → /ship
[ "$WRITES" -ge 2 ] && SUGGESTIONS="${SUGGESTIONS}  • /ship \"<message>\"\n      → when ready to
push\n"

[ -z "$SUGGESTIONS" ] && exit 0

{
  echo ""
  echo "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"
  echo "  Coach · suggestions"
  echo "━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"
  printf "%b" "$SUGGESTIONS"
```

```
  echo ""
} >&2

exit 0
```

Activate in `settings.json`:

.CLAUDE/SETTINGS.JSON

```
"Stop": [
  { "matcher": "*", "hooks": [
    { "type": "command", "command": ".claude/hooks/coach-suggest.sh" }
  ]}
]
```

Layer 3 — Manual `/coach` slash

For moments when you hesitate yourself. Create `.claude/commands/coach.md`:

.CLAUDE/COMMANDS/COACH.MD

```
---
description: "Analyze repo state and recommend next action"
---

# /coach

Run in parallel:
- `git status --short`, `git diff --stat`, `git log --oneline -5`
- Read `.claude/logs/activity.log | tail -50` if present

Compare state to Coach triggers in CLAUDE.md. Recommend max 3
prioritized actions:

  1. <command> → <reason in one sentence>
  2. <command> → <reason>
  3. <command> → <reason>

End with: "Run #1 or want to think first?"
```

The mute mechanism

Sometimes you know what you're doing and suggestions become noise. The hook checks a `.claude/coach-mute` flag and exits silently if it exists. Two commands to toggle:

- `/coach-mute` → `touch .claude/coach-mute` (Coach OFF)
- `/coach-on` → `rm -f .claude/coach-mute` (Coach ON)

> 💡 **NATURAL EVOLUTION**
>
> When you find yourself doing the same manual action 3 times, that's the signal to add a new Coach trigger. Open `CLAUDE.md` → Coach mode table → add a row. If you want the hook to detect it too, add the detection in `coach-suggest.sh`. That's how a conscious routine becomes an automatic reflex.

07 Maintenance, memory, MCP — beyond

Your setup is alive. CLAUDE.md goes stale, agents sleep, new MCPs appear. Here's the maintenance that keeps it healthy.

Persistent agent memory

By default, a sub-agent forgets everything between invocations. For critical agents (security audit, domain expert), you can enable cross-session memory versioned in git.

Pattern

1. Create the memory folder according to the desired scope (see table below)
2. Add `memory: project` (or `user`, or `local`) to the agent's frontmatter
3. Claude Code automatically injects read/write instructions into the sub-agent's system prompt, and activates Read/Write/Edit tools so it can manage its memory files

Three possible scopes:

SCOPE	LOCATION	WHEN
`user`	`~/.claude/agent-memory/<name>/`	Cross-project knowledge (personal preferences, reusable patterns)
`project`	`.claude/agent-memory/<name>/`	Project knowledge, versioned in git, shareable across team (recommended default)
`local`	`.claude/agent-memory-local/<name>/`	Project knowledge but sensitive / personal, gitignored

Typical memory folder structure:

STRUCTURE

```
.claude/agent-memory/security-auditor/
├── MEMORY.md              ← INDEX (first 200 lines or 25KB auto-loaded)
├── feedback_<topic>.md    ← User corrections / validations
├── project_<topic>.md     ← Decisions, deadlines, motivations
└── reference_<topic>.md   ← Pointers to external systems (Linear, Grafana…)
```

Typical benefit: a domain expert builds up a base of "validated formulas" or "project conventions" that survives re-clones and machine changes. No more re-debating each session.

MCP — connect Claude to the outside world

The **Model Context Protocol** gives Claude access to external tools (DB, browser, filesystem, APIs). Configure in `.mcp.json` at the root:

.MCP.JSON

```
{
  "mcpServers": {
    "playwright": {
      "command": "npx",
      "args": ["-y", "@playwright/mcp@latest", "--browser=chromium", "--isolated"]
    },
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "./data"]
    },
    "tavily": {
      "type": "http",
      "url": "https://mcp.tavily.com/mcp/",
      "headers": { "X-API-Key": "${TAVILY_API_KEY}" }
    }
  }
}
```

Which MCPs to install first

MCP	WHY	PRIORITY
Playwright	Test the UI in a browser. "Open the dashboard and check X."	essential for web apps
filesystem	Read access to a specific folder (CSV data, exports)	case-by-case
Tavily	Premium web search (better than default WebSearch)	if research is frequent
DB (Supabase, Postgres…)	Read DB schema without copy-paste — **lock read-only**	if DB is central

⚠️ **MCP SECURITY**

An MCP that can write to prod DB = potential catastrophe. Lock **read-only** wherever possible. Writes go through an explicit migration you review by hand.

Monthly setup audit

Every ~30 days, run a meta-audit to detect rot. Create `.claude/commands/audit-claude-setup.md` that checks:

- **Dormant agents**: grep `activity.log` for agents never invoked since creation
- **Stale references** in CLAUDE.md: files mentioned that no longer exist
- **Undocumented commands** in COMMANDS.md
- **Permission** `allow` entries never used (noise in settings.json)
- **Empty memories** for > 30 days (agent doesn't capitalize anything)
- **Slow hooks** (> 500 ms = friction)

Split CLAUDE.md when it grows

When CLAUDE.md exceeds ~100 lines, split into thematic files imported via `@`:

CLAUDE.MD

```
@AGENTS.md
@.claude/STACK.md
@.claude/CONVENTIONS.md
@.claude/WORKFLOW.md

# <Project> — configuration hub

| Looking for… | Go to |
|---|---|
| Commands + stack + gotchas | `@.claude/STACK.md` |
| Code conventions, dev rules, Git workflow | `@.claude/CONVENTIONS.md` |
| Agents, slash commands, Coach mode | `@.claude/WORKFLOW.md` |
| Copy-paste recipes | `.claude/PATTERNS.md` |
```

CLAUDE.md becomes a hub that points; thematic files stay < 80 lines each, readable at a glance.

Share the setup with a team

All `.claude/` versioned in git = every team member inherits automatically. To go further, package as a plugin:

STRUCTURE

```
.claude-plugin/<name>/
├── plugin.json            (metadata)
├── CLAUDE.md              (project brain)
├── settings.json.template (customizable template)
├── hooks.json             (references the hooks/ scripts)
├── agents/                (sub-agents)
├── skills/                (skills)
└── hooks/                 (scripts)
```

A plugin shares like a dependency. Useful when several internal projects share the same base setup.

08 Appendix — copy-paste templates

All templates from this guide, gathered and ready to paste. Adapt paths, project names, and stack to your context.

A. Universal `CLAUDE.md`

CLAUDE.MD

```
# <Project name>

One-sentence description.

## Stack
- Language / Framework / DB / Tests / Deployment

## Useful commands
- `<dev>`, `<verify>`, `<test>`, `<build>`

## Conventions
- Naming, structure, behavior

## Gotchas
- Known stack and project pitfalls

## Git workflow
- Branch, commit format, PR/push

## Hard rules
- Never X without confirmation
- Always Y before Z

## Coach mode (cf. chapter 6)
| Trigger | Action |
|---|---|
| ... | ... |
```

B. Complete `.claude/settings.json`

.CLAUDE/SETTINGS.JSON

```
{
  "$schema": "https://json.schemastore.org/claude-code-settings.json",
  "permissions": {
    "allow": [
      "Read", "Grep", "Glob",
      "Bash(git status:*)", "Bash(git diff:*)", "Bash(git log:*)",
      "Bash(git show:*)", "Bash(git branch:*)", "Bash(git fetch:*)",
      "Bash(ls:*)", "Bash(cat:*)", "Bash(find:*)",
      "Bash(head:*)", "Bash(tail:*)", "Bash(wc:*)"
```

```
    ],
    "deny": [
      "Bash(rm -rf*)", "Bash(rm)", "Bash(sudo*)",
      "Bash(curl* | bash*)"
    ]
  },
  "hooks": {
    "SessionStart": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/session-start.sh" }
      ]}
    ],
    "PreToolUse": [
      { "matcher": "Edit|Write|Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/pre-tool-guard.sh" }
      ]}
    ],
    "PostToolUse": [
      { "matcher": "Edit|Write", "hooks": [
        { "type": "command", "command": ".claude/hooks/post-edit-format.sh" },
        { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true }
      ]},
      { "matcher": "Bash", "hooks": [
        { "type": "command", "command": ".claude/hooks/activity-log.sh", "async": true }
      ]}
    ],
    "Stop": [
      { "matcher": "*", "hooks": [
        { "type": "command", "command": ".claude/hooks/coach-suggest.sh" }
      ]}
    ]
  }
}
```

C. activity-log hook (PostToolUse async)

.CLAUDE/HOOKS/ACTIVITY-LOG.SH

```
#!/usr/bin/env bash
# Zero-token lightweight log of Write/Edit/Bash to .claude/logs/activity.log
set -e
cd "$(dirname "$0")/../.."

INPUT="$(cat 2>/dev/null || echo '{}')"
TOOL=$(echo "$INPUT" | jq -r '.tool_name // empty' 2>/dev/null)
[ -z "$TOOL" ] && exit 0

LOG_DIR=".claude/logs"
LOG_FILE="$LOG_DIR/activity.log"
mkdir -p "$LOG_DIR"

TS=$(date '+%Y-%m-%d %H:%M:%S')
SID=$(echo "$INPUT" | jq -r '.session_id // empty' | cut -c1-8)
[ -z "$SID" ] && SID="--------"

case "$TOOL" in
  Bash)
    CMD=$(echo "$INPUT" | jq -r '.tool_input.command // empty' | tr '\n' ' ' | cut -c1-200)
    echo "[$TS] [$SID] BASH   | $CMD" >> "$LOG_FILE" ;;
  Write)
```

```
    FP=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
    echo "[$TS] [$SID] WRITE  | $FP" >> "$LOG_FILE" ;;
  Edit)
    FP=$(echo "$INPUT" | jq -r '.tool_input.file_path // empty')
    echo "[$TS] [$SID] EDIT   | $FP" >> "$LOG_FILE" ;;
esac

# Rotation: if > 5000 lines, keep last 3000
LINES=$(wc -l < "$LOG_FILE" 2>/dev/null || echo 0)
if [ "$LINES" -gt 5000 ]; then
  tail -3000 "$LOG_FILE" > "$LOG_FILE.tmp" && mv "$LOG_FILE.tmp" "$LOG_FILE"
fi

exit 0
```

D. pre-tool-guard hook (defensive PreToolUse)

See chapter 3 above. Don't forget `chmod +x` on all hooks.

E. session-start hook (SessionStart recap)

See chapter 3 above.

F. `security-auditor` sub-agent

.CLAUDE/AGENTS/SECURITY-AUDITOR.MD

```
---
name: security-auditor
description: >
  READ-ONLY security audit. Use proactively when user says
  "audit security", "GDPR", "check leaks", or before a public deployment.
  Covers: auth, secrets, injection, XSS, PII exposure, OWASP Top 10,
  basic GDPR compliance. Modifies no files.
model: opus
tools: Read, Grep, Glob, Bash, WebFetch, WebSearch
---

You are a SaaS app security auditor.

## Checklist (run systematically)

### 1. Auth & sessions
- User verified server-side before resource access?
- Server actions / API routes scope to user_id?
- Tokens in HttpOnly+Secure cookies? Refresh?

### 2. Row-Level Security (if Postgres/Supabase)
- For each sensitive table: policy "user only accesses own X"?
- INSERT/UPDATE/DELETE policies present (not just SELECT)?
- RLS enabled on ALL sensitive tables (otherwise = critical flaw)?

### 3. Injection / Validation
- User input inserted as-is in DB?
- Server-side validation (Zod / Pydantic)?
```

```
- File upload: mime/size limit, antivirus?

### 4. XSS / CSRF
- `dangerouslySetInnerHTML` justified?
- External links: `rel="noopener noreferrer"`?

### 5. Secrets & exposure
- `.env*` in `.gitignore`?
- Hardcoded API keys? (`grep -r "sk-"`, `grep -r "API_KEY"`)
- `NEXT_PUBLIC_*` contains only public data?

### 6. GDPR (if EU users)
- Consent collected? ToS / Privacy policy?
- Export rights? Right to erasure? Cascade delete?

### 7. Headers & infra
- HSTS, CSP, X-Content-Type-Options?

## Output format
For EACH finding:
- **Severity**: 🔴 / 🟠 / 🟡 / 🟢
- **File:line** or location (e.g. "users table")
- **Risk**: concrete consequence
- **Recommended fix**: code/SQL (don't apply)

End with: Score /10 + Top 3 urgent actions.

## Rules
- Distinguish "blocking for public" vs "before scale".
- If critical exploitable flaw → mention first.
- Read-only strict. No automatic fixes.
```

G. Complete `/ship` slash command

See chapter 5 above.

"30 minutes to start" checklist

STEP	ACTION	TIME
1	Install Claude Code (`curl -fsSL https://claude.ai/install.sh | bash` or Homebrew, or IDE extension)	2 min
2	Create `CLAUDE.md` at the root (template A)	10 min
3	Create `.claude/settings.json` with permissions (template B partial)	5 min
4	Update `.gitignore` (whitelist .claude/)	2 min
5	Test: ask Claude a question about the project stack	3 min
6	Optional: create defensive PreToolUse hook (template D)	8 min
Total	**≈ 30 minutes for a transformed Claude**	

The Claude Code Handbook — Volume 1. Living document: V2 will add "shareable team plugins" and "CI/CD integration". Feedback welcome. Written so we stop wasting time re-explaining to Claude what we already told it yesterday.